Dimensional Shift: Pulse-Driven Dimensional Computing and a Dimensional ADC Architecture

Bc. Ji■í Burda
Faculty of Applied Informatics (FAI), Tomas Bata University in Zlín (UTB), Czech Republic
Date: 2026-01-24

Abstract—

Dimensional Shift (DS) is a pulse-driven computing approach that alternates between (i) an EXPAND phase that explores multiple high-dimensional representations, configurations, or hypotheses, and (ii) a COLLAPSE phase that selects and compresses to a small, high-utility subset of dimensions (Dim-plosion). We present DS as a backend-agnostic architecture that can run on classical CPUs, accelerate decision roles on GPUs via a co-processor bridge (D-Link), and remain conceptually compatible with future quantum or hybrid backends. Building on this pulse model, we introduce a Dimensional Analog-to-Digital Converter (Dimensional ADC): a multidimensional encode–score–survive–decode pipeline that converts arbitrary signals into pulse-coded lane activity and reconstructs task-relevant outputs from a dynamically selected subset. A reference prototype (Project D-Link) demonstrates DS in three settings: (1) a dynamic 10-dimensional optimizer with polygon/radar visualization, (2) a CUDA primitives suite (vector addition, matrix multiplication, and RGB convolution) where DS performs configuration auto-tuning via EXPAND→COLLAPSE, and (3) scaling benchmarks for lane scoring and Top-K selection. We discuss robustness properties, limitations, and how DS relates to classical and quantum computing.

Index Terms—

Dimensional computing, pulse optimization, dimensionality reduction, GPU co-processor, Cross-Entropy Method, sampling-based MPC, compressive sensing, random projections, CUDA

I. Introduction

Classical digital computation is commonly organized as a fixed pipeline over binary state, while modern machine learning expands computation into continuous, high-dimensional spaces. Dimensional Shift (DS) proposes a unifying architectural viewpoint: treat dimensionality itself as a controllable resource, and compute in a rhythmic pulse of expansion and reduction.

The DS pulse is inspired by biological motifs (breathing, heartbeat) and by optimization loops that repeatedly generate candidates and retain elites. In DS, the EXPAND phase generates candidate representations or configurations; the COLLAPSE phase performs dimensional selection and compression (Dim-plosion), returning a compact state that drives the next pulse.

This paper contributes: (a) a formalization of DS as pulse-driven dimensional control; (b) a Dimensional ADC design that converts signals into multidimensional pulse codes and reconstructs outputs using adaptive lane survival; (c) a modular co-processor interface (D-Link) enabling CPU/CUDA execution; and (d) a prototype evaluation package (Project D-Link) supporting reproducible demonstrations for GPU systems.

II. Background and Related Work

DS connects to several established ideas:

• Elite-based stochastic optimization. The Cross-Entropy (CE) method iteratively samples candidates and updates a sampling distribution using elite sets, providing a canonical “generate → select elites → refit” loop.

• Sampling-based MPC. Information-theoretic MPC and related MPPI approaches use sampling to explore control sequences and weight or select them by cost.
• Dimensionality reduction. Random projections and the Johnson–Lindenstrauss (JL) lemma motivate preserving geometry under lower-dimensional embeddings; Achlioptas-style sparse random projections provide database-friendly implementations.
• Signal acquisition and compressive sensing. Compressive sensing shows that signals with structure can be recovered from fewer measurements than Nyquist sampling under suitable measurement operators and reconstruction.
• Spiking and pulse-based computation. Spiking neural models treat spike timing as information-bearing, motivating pulse-coded representations and event-driven processing.
• GPU performance engineering. CUDA programming guidelines emphasize memory locality; shared-memory tiling accelerates convolution by reducing redundant global memory traffic.
DS is not claimed to be equivalent to any single method; instead it is an architectural composition that uses these loops as building blocks and makes the pulse explicit as a system-level control policy.

III. Dimensional Shift (DS) Computing Model

A DS system maintains a state vector of parameters and a set of active dimensions ("lanes"). Each pulse alternates:
A) EXPAND (DE). Explore candidate representations/configurations. Examples: sampling alternative lane subsets, sampling thresholds, proposing kernel launch parameters, or evaluating multiple projections.
B) COLLAPSE (DR / Dim-plosion). Select a compact subset that best satisfies an objective and stability constraints, then commit to it for the next pulse.
Objective. DS optimizes a composite cost, typically: $J = E + \alpha_t \cdot T + \alpha_s \cdot ||\Delta p||$, where E is task error (e.g., reconstruction RMSE, control cost), T is compute time (latency/energy proxy), and p are DS parameters (lanes, Top-K size, pulse times Te/Tc, thresholds). The stability term discourages oscillatory control.
Mapping to "dimensions." In DS, each additional controllable parameter can be treated as a dimension in the meta-optimization space. In practice, DS distinguishes between (i) signal dimensions (lanes used to encode/compute) and (ii) policy dimensions (parameters controlling the pulse).

IV. Dimensional ADC Architecture

We define a Dimensional ADC (D-ADC) as a multidimensional signal conversion pipeline:
1) Encode. Convert a signal window x(t) into lane activity b(t) using projections, thresholds, or pulse-time encoding. A simple form uses a history matrix H and lane projections W: $z = W \cdot H$, $b = 1[z > \tau]$. 2) Score (Insight). Evaluate lane utility scores s_i that reflect predictive value, novelty, energy, or task relevance. 3) Survive (Dim-plosion). Retain Top-K lanes (or an elite set) and discard the remainder for the next pulse. 4) Decode. Reconstruct x■(t) (or a task output y■) from the reduced code, e.g., via linear decoding or learned decoding.
This pipeline generalizes classical ADC: instead of mapping analog amplitude to a fixed binary word, D-ADC maps signals into a dynamic high-dimensional pulse code whose effective resolution adapts through survival selection. In the prototype, D-ADC is demonstrated via DS-lite encoding/decoding with parameter evolution and visualization.

V. D-Link: A DS Co-Processor Bridge (CPU/CUDA/QPU-Ready)

D-Link is an execution router for DS "roles": lane scoring (insight_eval), Top-K selection (topk_subset), and gate/pulse timing optimization (gate_opt). The DS controller emits a role problem; D-Link selects a backend (CPU or CUDA today) and returns results.

On GPUs, the key advantage is that scoring and selection over many lanes are embarrassingly parallel. By treating these roles as co-processor functions, DS can adapt its internal dimensions and policies at high frequency without blocking the main application loop.

The prototype includes: • CPU backend for portability and correctness baselines. • CUDA backend (via CuPy in Python demos) for lane-scale acceleration when available. • A conceptual placeholder interface for future quantum or hybrid backends: DS roles are expressed as problem dictionaries, enabling later mapping to QPU routines without changing the DS controller's API.

VI. Prototype Demonstrations (Project D-Link)

Project D-Link provides three demonstrators:

A) Dynamic 10D DS optimizer. A 10-dimensional parameter vector evolves across pulses; polygon/radar plots visualize parameter trajectories. The demo compares DS to baselines under a composite objective.

B) UTB CUDA primitives suite with DS pulse. A C/CUDA application implements vector addition, matrix multiplication (global-memory baseline), and RGB convolution with both global-memory and shared-memory tiled implementations. DS is used as an autotuning pulse: EXPAND benchmarks candidate block sizes/configurations; COLLAPSE runs the best configuration. A one-command benchmark script generates CSV timing, plots, and a markdown report explaining why shared memory improves convolution throughput.

C) D-Link scaling benchmark. The benchmark measures role execution time as lane count increases (e.g., 10k→1M lanes) and compares CPU vs CUDA (when available). Outputs include CSV and plots for insight_eval and topk_subset.

VII. Discussion: Robustness, Limits, and Relation to Quantum Computing

Robustness. DS is robust in the architectural sense that it explicitly manages representation capacity (dimensions) and policy timing. When signals or workloads shift, DS can re-expand to search and then re-collapse to a stable compact policy. Logging and stability penalties make the controller auditable and reproducible.

Limits. DS EXPAND has overhead: exploring candidates costs time and energy. DS is advantageous when (i) workloads are nonstationary, (ii) the selected policy can be reused across many steps, or (iii) parallel hardware makes exploration cheap (e.g., GPUs). DS is not a substitute for a highly tuned fixed kernel in a stationary regime.

Relation to quantum computing. DS is not "the same as quantum computing" in the strict physical sense: quantum advantage depends on amplitude interference and entanglement. However, DS shares a useful metaphor: EXPAND resembles generating a superposition of hypotheses (classically, by sampling), while COLLAPSE resembles selection/measurement. A precise claim is: DS is a classical pulse-driven dimensional optimization architecture that can be mapped onto GPU and, in principle, to hybrid QPU backends for certain roles, but it does not automatically inherit quantum speedups.

VIII. Conclusion

Dimensional Shift reframes computation as a pulse-driven control problem over dimensional resources. The Dimensional ADC provides a concrete signal-processing instance: encode into many lanes, score, select survivors, and decode. The D-Link bridge makes DS practical by routing DS roles to CPU/CUDA backends and producing reproducible artifacts (CSV, plots, reports). The prototype package demonstrates DS on CUDA primitives and dynamic dimensional optimization, and provides a pathway for future work: strict PNG/libpng++ integration, large-scale lane batching, hybrid QPU role execution, and task-driven evaluations in adaptive sensing, control, and continual learning.

Acknowledgment

This manuscript accompanies the open-source prototype package “Project D-Link” (Dimensional Shift).

References

[1] R. Y. Rubinstein and D. P. Kroese, The Cross-Entropy Method: A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation, and Machine Learning. New York, NY, USA: Springer, 2004.
[2] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, “Information Theoretic Model Predictive Control: Theory and Applications to Autonomous Driving,” arXiv:1707.02342, 2017.
[3] D. L. Donoho, “Compressed Sensing,” IEEE Trans. Inf. Theory, vol. 52, no. 4, pp. 1289–1306, 2006.
[4] W. B. Johnson and J. Lindenstrauss, “Extensions of Lipschitz mappings into a Hilbert space,” in Conference in Modern Analysis and Probability, Contemporary Mathematics, 1984.
[5] D. Achlioptas, “Database-friendly random projections: Johnson–Lindenstrauss with binary coins,” J. Comput. Syst. Sci., vol. 66, no. 4, pp. 671–687, 2003.
[6] W. Maass, “Networks of spiking neurons: The third generation of neural network models,” Neural Networks, vol. 10, no. 9, pp. 1659–1671, 1997.
[7] NVIDIA, CUDA C++ Programming Guide, Release 13.1, 2025.
[8] M. A. Nielsen and I. L. Chuang, Quantum Computation and Quantum Information: 10th Anniversary Edition. Cambridge, U.K.: Cambridge Univ. Press, 2010.
[9] UL HPC Tutorials, “CUDA Convolution using GPU (shared memory best practices),” online tutorial, accessed 2026.